

```
import javax.swing.*;

import javax.swing.table.DefaultTableCellRenderer;

import javax.swing.table.DefaultTableModel;

import java.awt.*;

import java.util.*;

import java.util.List;


public class HospitalQueueSystem {

    private JFrame frame;

    private JTable queueTable;

    private DefaultTableModel tableModel;

    private ArrayList<Patient> patients;

    private int queueNumber = 1;

    private JLabel statusLabel;


    // Condition categories with priorities
    private enum ConditionCategory {

        INTERNAL_DAMAGE_EMERGENCY("Internal Damage - Emergency", 1, 5),

        INTERNAL_DAMAGE_NORMAL("Internal Damage - Normal", 2, 15),

        PHYSICAL_DAMAGE_EMERGENCY("Physical Damage - Emergency", 3, 10),

        PHYSICAL_DAMAGE_NORMAL("Physical Damage - Normal", 4, 30);

        private final String displayName;

        private final int priority;

        private final int estimatedTime; // in minutes
```

```
ConditionCategory(String displayName, int priority, int estimatedTime) {  
    this.displayName = displayName;  
    this.priority = priority;  
    this.estimatedTime = estimatedTime;  
}  
  
public String getDisplayName() { return displayName; }  
public int getPriority() { return priority; }  
public int getEstimatedTime() { return estimatedTime; }  
}
```

// Patient class

```
class Patient {  
    int queueNo;  
    String name;  
    ConditionCategory condition;  
    String status;  
    String eta;  
    boolean isEmergency;  
    Date arrivalTime;  
  
    Patient(int queueNo, String name, ConditionCategory condition) {  
        this.queueNo = queueNo;  
        this.name = name;  
        this.condition = condition;  
        this.status = "Waiting";  
    }  
}
```

```
this.isEmergency = condition.getPriority() <= 2; // Emergency if priority 1 or 2  
this.arrivalTime = new Date();  
this.eta = calculateETA();  
}
```

```
private String calculateETA() {  
    int totalWaitTime = 0;  
  
    // Calculate wait time based on patients ahead with higher or equal priority  
    for (Patient p : patients) {  
        if (p.status.equals("Waiting") && p.condition.getPriority() <=  
this.condition.getPriority()) {  
            totalWaitTime += p.condition.getEstimatedTime();  
        }  
    }  
  
    // Add current patient's estimated time  
    totalWaitTime += this.condition.getEstimatedTime();  
  
    if (totalWaitTime < 60) {  
        return totalWaitTime + " min";  
    } else {  
        int hours = totalWaitTime / 60;  
        int minutes = totalWaitTime % 60;  
        return hours + "h " + minutes + "m";  
    }  
}
```

```
}
```

```
public void updateETA() {  
    this.eta = calculateETA();  
}
```

```
}
```

```
public HospitalQueueSystem() {  
    patients = new ArrayList<>();  
    initializeGUI();  
}
```

```
private void initializeGUI() {  
    // Create main frame  
    frame = new JFrame("Hospital Queue Management");  
    frame.setDefaultCloseOperation(JFrame.EXIT_ON_CLOSE);  
    frame.setSize(1000, 700);  
    frame.setLayout(new BorderLayout());  
  
    // Title  
    JLabel titleLabel = new JLabel("HOSPITAL QUEUE MANAGEMENT SYSTEM",  
JLabel.CENTER);  
    titleLabel.setFont(new Font("Serif", Font.BOLD, 28));  
    titleLabel.setForeground(Color.BLUE);  
    titleLabel.setBorder(BorderFactory.createEmptyBorder(10, 0, 10, 0));  
    frame.add(titleLabel, BorderLayout.NORTH);
```

```
// Main panel

JPanel mainPanel = new JPanel(new BorderLayout());

// Table for displaying queue

String[] columns = {"Queue No", "Patient Name", "Condition", "Emergency", "Status",
"ETA"};

tableModel = new DefaultTableModel(columns, 0) {

    public boolean isCellEditable(int row, int column) {

        return false;

    }

    @Override

    public Class<?> getColumnClass(int columnIndex) {

        return String.class;

    }

};

queueTable = new JTable(tableModel);

queueTable.setSelectionMode(ListSelectionModel.SINGLE_SELECTION);

queueTable.setRowHeight(25);

queueTable.getTableHeader().setFont(new Font("SansSerif", Font.BOLD, 12));

// Set custom renderer for color coding

queueTable.setDefaultRenderer(Object.class, new CustomTableCellRenderer());

JScrollPane tableScroll = new JScrollPane(queueTable);
```

```
tableScroll.setPreferredSize(new Dimension(900, 400));
```

```
// Button panel
```

```
JPanel buttonPanel = new JPanel(new FlowLayout());
```

```
buttonPanel.setBorder(BorderFactory.createEmptyBorder(10, 0, 10, 0));
```

```
JButton addButton = new JButton("Add Patient");
```

```
JButton callButton = new JButton("Call Next");
```

```
JButton completeButton = new JButton("Complete");
```

```
JButton removeButton = new JButton("Remove");
```

```
JButton refreshButton = new JButton("Refresh");
```

```
JButton freezeButton = new JButton("Freeze Emergency");
```

```
// Style buttons
```

```
Color buttonColor = new Color(70, 130, 180);
```

```
Color emergencyColor = new Color(220, 20, 60);
```

```
addButton.setBackground(buttonColor);
```

```
addButton.setForeground(Color.WHITE);
```

```
callButton.setBackground(buttonColor);
```

```
callButton.setForeground(Color.WHITE);
```

```
completeButton.setBackground(buttonColor);
```

```
completeButton.setForeground(Color.WHITE);
```

```
removeButton.setBackground(buttonColor);
```

```
removeButton.setForeground(Color.WHITE);
```

```
refreshButton.setBackground(buttonColor);
```

```
refreshButton.setForeground(Color.WHITE);

freezeButton.setBackground(emergencyColor);
freezeButton.setForeground(Color.WHITE);


// Add buttons to panel
buttonPanel.add(addButton);
buttonPanel.add(callButton);
buttonPanel.add(completeButton);
buttonPanel.add(removeButton);
buttonPanel.add(refreshButton);
buttonPanel.add(freezeButton);


// Add components to main panel
mainPanel.add(tableScroll, BorderLayout.CENTER);
mainPanel.add(buttonPanel, BorderLayout.SOUTH);


frame.add(mainPanel, BorderLayout.CENTER);


// Status panel
JPanel statusPanel = new JPanel(new FlowLayout());
statusLabel = new JLabel("Total Patients: 0 | Waiting: 0 | In Consultation: 0 | Completed:
0");
statusPanel.add(statusLabel);
frame.add(statusPanel, BorderLayout.SOUTH);


// Button actions
```

```

addButton.addActionListener(e -> addPatient());
callButton.addActionListener(e -> callNextPatient());
completeButton.addActionListener(e -> completePatient());
removeButton.addActionListener(e -> removePatient());
refreshButton.addActionListener(e -> refreshTable());
freezeButton.addActionListener(e -> freezeEmergencyPatients());

frame.setVisible(true);
}

// Custom table cell renderer for color coding
public final class CustomTableCellRenderer extends DefaultTableCellRenderer {
    @Override
    public Component getTableCellRendererComponent(JTable table, Object value,
                                                    boolean isSelected, boolean hasFocus, int row, int column) {
        Component c = super.getTableCellRendererComponent(table, value,
                                                            isSelected, hasFocus, row, column);

        String status = table.getValueAt(row, 4).toString();
        boolean isEmergency = "Yes".equals(table.getValueAt(row, 3).toString());

        // Set background colors based on status and emergency
        if (isSelected) {
            c.setBackground(new Color(173, 216, 230)); // Light blue for selection
        } else if (isEmergency) {
            c.setBackground(new Color(255, 182, 193)); // Light red for emergency
        }
    }
}

```



```

    } else {
        switch (status) {
            case "Waiting":
                c.setBackground(Color.YELLOW);
                break;
            case "In Consultation":
                c.setBackground(Color.ORANGE);
                break;
            case "Completed":
                c.setBackground(new Color(144, 238, 144)); // Light green
                break;
            default:
                c.setBackground(Color.WHITE);
        }
    }

    return c;
}
}

```

```

private void addPatient() {
    // Create dialog for patient details

    JTextField nameField = new JTextField(20);

    // Condition selection with categories

    JComboBox<String> conditionComboBox = new JComboBox<>();
}

```

```
for (ConditionCategory category : ConditionCategory.values()) {  
    conditionComboBox.addItem(category.getDisplayName());  
}
```

```
JPanel panel = new JPanel(new GridLayout(2, 2, 10, 10));  
panel.setBorder(BorderFactory.createEmptyBorder(10, 10, 10, 10));  
panel.add(new JLabel("Patient Name:"));  
panel.add(nameField);  
panel.add(new JLabel("Condition:"));  
panel.add(conditionComboBox);
```

```
int result = JOptionPane.showConfirmDialog(frame, panel,  
    "Add New Patient", JOptionPane.OK_CANCEL_OPTION,  
    JOptionPane.PLAIN_MESSAGE);
```

```
if (result == JOptionPane.OK_OPTION) {  
    String name = nameField.getText().trim();  
    String selectedCondition = (String) conditionComboBox.getSelectedItem();
```

```
if (!name.isEmpty() && selectedCondition != null) {  
    // Find the condition category  
    ConditionCategory condition = null;  
    for (ConditionCategory cat : ConditionCategory.values()) {  
        if (cat.getDisplayName().equals(selectedCondition)) {  
            condition = cat;  
            break;
```

```

    }
}

if (condition != null) {
    Patient patient = new Patient(queueNumber, name, condition);

    // Insert patient based on priority
    insertPatientByPriority(patient);

    queueNumber++;
    refreshTable();

    String message = "Patient added! Queue Number: " + patient.queueNo;
    if (patient.isEmergency) {
        message += " (EMERGENCY - Priority " + condition.getPriority() + ")";
    }
    JOptionPane.showMessageDialog(frame, message);
}
} else {
    JOptionPane.showMessageDialog(frame,
        "Please enter both name and select condition!");
}
}
}

private void insertPatientByPriority(Patient newPatient) {

```

```

int insertIndex = patients.size();

for (int i = 0; i < patients.size(); i++) {
    Patient existingPatient = patients.get(i);

    // Insert before patients with lower priority (higher priority number)
    if (existingPatient.condition.getPriority() > newPatient.condition.getPriority()
        && existingPatient.status.equals("Waiting")) {
        insertIndex = i;
        break;
    }
}

patients.add(insertIndex, newPatient);
updateAllETAs();
}

private void updateAllETAs() {
    for (Patient patient : patients) {
        if (patient.status.equals("Waiting")) {
            patient.updateETA();
        }
    }
}

private void callNextPatient() {

```

```
if (patients.isEmpty()) {  
    JOptionPane.showMessageDialog(frame, "No patients in queue!");  
    return;  
}
```

```
// Find first waiting patient (highest priority due to sorted insertion)
```

```
for (Patient patient : patients) {  
    if (patient.status.equals("Waiting")) {  
        patient.status = "In Consultation";  
        updateAlLETAs();  
        refreshTable();  
        JOptionPane.showMessageDialog(frame,  
            "Calling: " + patient.name + " (Queue: " + patient.queueNo +  
            " - " + patient.condition.getDisplayName() + ")");  
        return;  
    }  
}
```

```
JOptionPane.showMessageDialog(frame, "No patients waiting!");  
}
```

```
private void completePatient() {  
    int selectedRow = queueTable.getSelectedRow();  
    if (selectedRow == -1) {  
        JOptionPane.showMessageDialog(frame, "Please select a patient first!");  
        return;  
    }  
}
```

```

    }

    int queueNo = (int) tableModel.getValueAt(selectedRow, 0);
    for (Patient patient : patients) {
        if (patient.queueNo == queueNo) {
            patient.status = "Completed";
            patient.eta = "Completed";
            updateAllETAs();
            refreshTable();
            JOptionPane.showMessageDialog(frame,
                "Completed: " + patient.name);
            return;
        }
    }
}

```

```

private void removePatient() {
    int selectedRow = queueTable.getSelectedRow();
    if (selectedRow == -1) {
        JOptionPane.showMessageDialog(frame, "Please select a patient first!");
        return;
    }
}

```

```

int queueNo = (int) tableModel.getValueAt(selectedRow, 0);
patients.removeIf(patient -> patient.queueNo == queueNo);
updateAllETAs();

```

```
refreshTable();  
JOptionPane.showMessageDialog(frame, "Patient removed from queue");  
}
```

```
private void freezeEmergencyPatients() {  
    int frozenCount = 0;  
  
    // Move all emergency patients to the front  
    ArrayList<Patient> emergencyPatients = new ArrayList<>();  
    ArrayList<Patient> normalPatients = new ArrayList<>();  
  
    for (Patient patient : patients) {  
        if (patient.isEmergency && patient.status.equals("Waiting")) {  
            emergencyPatients.add(patient);  
            frozenCount++;  
        } else {  
            normalPatients.add(patient);  
        }  
    }  
}
```

```
// Rebuild patients list with emergencies first  
patients.clear();  
patients.addAll(emergencyPatients);  
patients.addAll(normalPatients);  
  
updateAllETAs();
```

```
refreshTable();
```

```
JOptionPane.showMessageDialog(frame,  
    "Emergency patients prioritized! " + frozenCount + " emergency cases moved to  
front.");  
}
```

```
private void refreshTable() {  
    // Clear table  
    tableModel.setRowCount(0);  
  
    // Add all patients to table  
    for (Patient patient : patients) {  
        Object[] row = {  
            patient.queueNo,  
            patient.name,  
            patient.condition.getDisplayName(),  
            patient.isEmergency ? "Yes" : "No",  
            patient.status,  
            patient.eta  
        };  
        tableModel.addRow(row);  
    }  
  
    // Update status  
    updateStatus();  
}
```



```
}
```

```
private void updateStatus() {
```

```
    int waiting = 0;
```

```
    int inConsultation = 0;
```

```
    int completed = 0;
```

```
    int emergency = 0;
```

```
    for (Patient patient : patients) {
```

```
        switch (patient.status) {
```

```
            case "Waiting": waiting++; break;
```

```
            case "In Consultation": inConsultation++; break;
```

```
            case "Completed": completed++; break;
```

```
        }
```

```
        if (patient.isEmergency && !patient.status.equals("Completed")) {
```

```
            emergency++;
```

```
        }
```

```
    }
```

```
    statusLabel.setText(String.format(
```

```
        "Total: %d | Waiting: %d | In Consultation: %d | Completed: %d | Emergency Cases: %d",
```

```
        patients.size(), waiting, inConsultation, completed, emergency
```

```
    ));
```

```
}
```

```
public static void main(String[] args) {  
    // Run the application  
    SwingUtilities.invokeLater(() -> new HospitalQueueSystem());  
}  
}
```